A Computer-Assisted Counterexample to the 22-Color Conjecture for Bridge-Connected K_6 Chains

Aurantium

Abstract

Let H_t consist of t vertex-disjoint copies of K_6, with a single bridge between each consecutive pair and distinct left and right bridge vertices within every clique. We give a computer-assisted proof that $\chi_{\mathrm{p}}(H_t) \leq 23$ for every $t \geq 1$ and $\chi_{\mathrm{p}}(H_t) = 23$ for every $t \geq 75$. In particular, the 450-vertex graph H_{75} disproves Conjecture 2 of Furmańczyk, Gözüpek and Özkan (arXiv:2511.12761v1), and 23 is the sharp uniform upper bound for this family. For the lower bound, every 22-coloring yields a necessary scheduling relaxation. An integer decreasing-rank certificate with 16,646,562 records rules out a 75-row schedule; two checker implementations have each checked all 1,755,492,299 labeled choices. For the upper bound, an explicit 480-clique periodic coloring is verified using graph distances. The proof uses exact integer checks and does not require trusting the certificate generator. We do not establish that 75 is the least counterexample length.

1 The graph family and main result

We work with finite, simple, undirected, unweighted graphs in the usual ZFC foundations. A *packing k-coloring* of a graph G is a map $c : V(G) \to \{1, \ldots, k\}$ such that distinct vertices of color i have graph distance strictly greater than i. The packing chromatic number $\chi_{\mathrm{p}}(G)$ is the least such k. The numerical color labels matter: an arbitrary permutation of them need not preserve a packing coloring.

For each integer $t \geq 1$, let H_t have pairwise disjoint vertex sets

$$V_j = \{L_j, A_j, B_j, C_j, D_j, R_j\}, \qquad 1 \leq j \leq t.$$

Each V_j induces a K_6. The only edges between different cliques are R_jL_{j+1} for $1 \le j < t$. In particular, $L_j \neq R_j$, and neighboring cliques do not share vertices. The proposed uniform 22-color bound is

$$\begin{aligned} &\forall t \ge 1 \quad \exists c : V(H_t) \longrightarrow \{1, \dots, 22\}, \\ &u \neq v, \quad c(u) = c(v) = i \quad \Longrightarrow \quad d_{H_t}(u, v) > i. \end{aligned} \tag{T}$$

This is an upper-bound assertion; it does not assert that every finite chain has packing chromatic number exactly 22.

The assertion comes from Conjecture 2 in Section 3.2.1 of Furmańczyk, Gözüpek and Özkan [1, PDF p. 12]. Their Definition 2 partitions a path into vertex-disjoint blocks, each identified with a path inside a copy of the prescribed graph [1, PDF p. 3]. For K_6 and $2 \le \ell \le 6$, the first and last vertices of each ℓ-vertex block are distinct. Name them L_j and R_j. All internal block edges are already clique edges, and the only edges between consecutive blocks are the bridges R_jL_{j+1}. Conversely, in every clique of H_t, choose any simple ℓ-vertex path from L_j to R_j. This realizes the original construction, so

$$H_t \cong P_{\ell t} \diamond_\ell K_6 \qquad (2 \le \ell \le 6).$$

This is also the relevant specialization of Proposition 20 [1, PDF p. 10]. Thus the graph family here is the one in the conjecture, with bridge-connected, vertex-disjoint cliques.

Theorem 1. *For the graph family above,*

$$\chi_{\mathrm{p}}(H_t) \le 23 \quad (t \ge 1), \qquad \chi_{\mathrm{p}}(H_t) = 23 \quad (t \ge 75).$$

In particular, $\chi_{\mathrm{p}}(H_{75}) = 23$*, and 23 is the sharp uniform upper bound for the family.*

The graph H_{75} has 450 vertices and 1199 edges. Its complete edge list and a 23-coloring are supplied in `certificates/H75_graph_and_23color_witness.json`. The impossibility of a 22-coloring follows from the necessary relaxation, rank induction, and complete finite certificate below. It is not an inference from an unsuccessful periodic search or an exclusion restricted to a paired-color template. We do not prove that 75 is the smallest counterexample length or determine all packing chromatic numbers for $t < 75$.

2 Boundary cases

Since $H_1 = K_6$, we have $\chi_{\mathrm{p}}(H_1) = 6$. The diameter of H_2 is 3. Each color at least 3 can therefore occur at most once, while each of colors 1 and 2 can

occur at most twice. Nine colors can cover at most $2+2+7=11$ vertices, fewer than the 12 vertices of H_2. In the order (L, A, B, C, D, R), the two rows

$$(3,1,2,4,5,6), \qquad (7,1,2,8,9,10)$$

give a packing 10-coloring. Consequently $\chi_{\mathrm{p}}(H_2) = 10$.

3 Distances and a necessary scheduling relaxation

Lemma 1 (Exact distances)**.** *If $x \in V_j$, $y \in V_{j+r}$, and $r \geq 1$, then*

$$d(x,y) = 2r - 1 + \mathbf{1}[x \neq R_j] + \mathbf{1}[y \neq L_{j+r}]. \tag{1}$$

Proof. Every path between the two vertices must traverse r bridges. Within each of the $r-1$ strictly intermediate cliques, it must also travel from the left bridge vertex to the distinct right bridge vertex, using at least one internal edge. The initial and final cliques contribute the two endpoint costs in (1). Since every clique is complete, these lower bounds can all be attained simultaneously. Leaving this segment of the bridge chain and returning cannot shorten a path. □

In particular, $d(x,y) \leq 2r+1$. Set

$$q_i = \lceil i/2 \rceil.$$

If color i occurs in two cliques whose indices differ by $r < q_i$, then $2r+1 \leq i$, contradicting the packing condition. Occurrences of color i must therefore have clique indices differing by at least q_i. This is a necessary condition, not a sufficient one.

Every K_6 uses six distinct colors, at most two of which are 1 and 2. Suppose that a 22-coloring exists. In each clique, retain any four colors greater than 2 and discard all other occurrences. The result is a necessary schedule with t rows: each row selects four distinct tasks, corresponding to colors 3 through 22, and successive selections of the task for color i are separated by at least $\lceil i/2 \rceil$ rows. The waiting parameters of these twenty tasks are

$$2, 2, 3, 3, \ldots, 11, 11. \tag{2}$$

Deleting occurrences cannot reduce the gaps between retained occurrences. This model also discards every endpoint restriction, so it is weaker than the

original coloring problem. Hence

$$\begin{aligned} &H_t \text{ has a packing 22-coloring} \\ \Longrightarrow\ &\text{the tasks in (2) admit a legal } t\text{-row schedule} \\ &\text{with exactly four tasks selected in each row.} \end{aligned} \tag{3}$$

This argument does not assume that colors 1 and 2 occur in every clique. It also does not require colors $2q-1$ and $2q$ to occur together. Neither restriction is used in the proof.

4 Scheduling states and exact transitions

Immediately before a row, a task of waiting parameter q has a residual waiting time $r \in \{0, \dots, q-1\}$. It is available exactly when $r = 0$. A step selects four of the currently available task copies. A selected task has new residual $q-1$; an unselected task has new residual $\max(r-1, 0)$. This rule expresses exactly the requirement that selections of a task be separated by at least q rows.

The two tasks with the same parameter q are interchangeable within this necessary relaxation. We may therefore sort their residuals as $0 \le a_q \le b_q < q$. This sorting takes a quotient by the symmetry of task labels; it does not force the tasks to be selected together. Every labeled schedule projects to a sequence of these sorted states. The full state domain is

$$\mathcal{S} = \prod_{q=2}^{11} \{(a,b) : 0 \le a \le b < q\}, \qquad |\mathcal{S}| = \prod_{q=2}^{11} \frac{q(q+1)}{2} = 9\,336\,040\,560\,000.$$

The initial state s_0 has all twenty residuals equal to zero, because no selections precede the first row. The complete checkers enumerate every labeled four-element subset of available task copies at each listed state. Distinct labeled choices that become the same sorted successor are still covered individually.

5 Future-capacity inequalities

Lemma 2 (Capacity of a single task)**.** *A task with parameter q and current residual r can be selected at most $M_q(h, r)$ times in the next h rows, where*

$$M_q(h,r) = \begin{cases} 0, & h \le r, \\ 1 + \lfloor (h-1-r)/q \rfloor, & h > r \end{cases} \tag{4}$$

Proof. Number the future rows $0, \dots, h-1$. The earliest allowed row is r, and subsequent selections must be at least q rows apart. Thus the earliest possible selection times are $r, r+q, r+2q, \dots$. Counting the terms at most $h-1$ gives (4). □

For a state $s = ((a_q, b_q))_{q=2}^{11}$, define

$$C_h(s) = \sum_{q=2}^{11} \big(M_q(h, a_q) + M_q(h, b_q)\big).$$

Four tasks are needed in each row. It follows that

$$C_h(s) < 4h \quad \Longrightarrow \quad s \text{ has no legal extension of } h \text{ rows.} \tag{5}$$

This is an integer necessary inequality. It uses no numerical optimization, and it need not characterize every feasible state.

6 A finite decreasing-rank certificate

Lemma 3 (Decreasing ranks)**.** *Let $D \subseteq \mathcal{S}$ be finite with $s_0 \in D$, and let $R : D \to \mathbb{Z}_{>0}$. Suppose that, for every $s \in D$ and every legal one-step successor s', at least one of the following holds:*

(A) There is an integer h with $1 \le h < R(s)$ such that $C_h(s') < 4h$.

(B) $s' \in D$ and $R(s') < R(s)$.

Then no state $s \in D$ admits a legal schedule of length $R(s)$.

Proof. Use strong induction on the positive integer $R(s)$. Suppose a legal schedule of length $R(s)$ starts at s. After its first step, $R(s)-1$ rows remain. In case (A), the first h rows of this remaining schedule contradict (5). In case (B), the first $R(s')$ remaining rows contradict the induction hypothesis. If there is no legal first step, the conclusion is immediate. □

The lemma does not require D to contain every possible state. It also does not require trusting a search procedure: the required check is that all legal transitions from every listed state are covered by (A) or (B).

7 The complete lower-bound certificate

7.1 Records and exact encoding

The principal certificate is distributed as a separate compressed asset, `H75_rank_certificate.bin.gz`. Its decompressed path in the accompanying package is `certificates/full_fast50m.bin`. It consists of 16,646,562 records, each an eight-byte little-endian unsigned integer, with no header. Its total length is 133,172,496 bytes. Every recorded rank is positive, and the initial state has rank 75. The SHA-256 of the decompressed certificate is

`082c86b92fa216f25c8f4dcb2abf97bfa88c3f98ca93b2de150041e8eb5e4fb0`.

The compressed asset has 49,164,961 bytes and SHA-256

`eb42565bf1c6e3e0e732583c13564886a05416c889c8b00dcedffd9ce6ad94b6`.

Both identities are also specified in `CERTIFICATE_ASSET.json`.

For each q, put

$$B_q = \frac{q(q+1)}{2}, \qquad u_q = \frac{b_q(b_q+1)}{2} + a_q.$$

The mixed-radix state identifier is

$$\mathrm{id}(s) = \sum_{q=2}^{11} u_q \prod_{v=2}^{q-1} B_v,$$

where an empty product is 1. A 64-bit record w encodes

$$\mathrm{key} = w \bmod 2^{44} = \mathrm{id}(s) + 1, \qquad R(s) = \left\lfloor w/2^{44} \right\rfloor.$$

The records are ordered by strictly increasing key, and the initial state has key 1. Every valid state identifier is less than $2^{44} - 1$. To decode a state, recover successive digits u_q by Euclidean division by the radices B_q. The unique triangular interval

$$\frac{b(b+1)}{2} \le u_q < \frac{(b+1)(b+2)}{2}$$

determines $b_q = b$, and then $a_q = u_q - b(b+1)/2$. The full format and checker contract are given in `docs/CERTIFICATE_FORMAT.md`.

7.2 The checking algorithm

The following pseudocode specifies a direct complete check. Here $Q = (2, 2, 3, 3, \ldots, 11, 11)$, and the residuals in each equal-Q pair are kept in sorted order.

```
Read every little-endian record and validate its encoding,
positive rank, state domain, and strictly increasing key.
Store the resulting map R, and require R[all_zero] == 75.
For every state s in R:
    available = task copies with residual zero in s
    For every four-element subset A of available:
        t[i] = Q[i]-1 if i is in A
               else max(s[i]-1, 0)
        Sort each equal-Q pair in t.
        If 0 < R.get(t, 0) < R[s]:
            Accept this transition.
        Otherwise:
            Require some h in 1..min(72, R[s]-1) with
                sum(M(Q[i], h, t[i])) < 4*h.
Accept only if every transition passes.
```

The finite certificate metadata, including byte count, record count and root rank, must match the stated certificate. Restricting the capacity search to $h \leq 72$ is sound: a single obstruction with $1 \leq h < R(s)$ suffices. There is no assumption that this restricted search recognizes all impossible states. A transition without such an obstruction must have a listed successor of strictly smaller positive rank.

For efficient capacity checking, first decrease every residual to $\max(r - 1, 0)$, obtaining a base state b. If an available task with parameter q is selected, its residual changes from 0 in b to $q - 1$ in the successor. The resulting decrease in its h-row capacity is exactly $\mathbf{1}[q \nmid h]$. Thus, if the four selected copies form A, the successor satisfies

$$C_h(s') = C_h(b) - 4 + |\{a \in A : q_a \mid h\}|. \tag{6}$$

This identity follows directly by counting possible selection times $0, q, 2q, \ldots$ and $q-1, 2q-1, \ldots$. The implementations can test capacity before doing the rank lookup. The original checker also combines mathematically identical capacity tests; neither optimization changes the obligation to cover every labeled choice.

The package contains the original complete checker, `code/verify_full_rank.cpp`; a portable copy, `code/verify_full_rank_portable.cpp`; and a

separately implemented checker, `code/independent_rank_check.cpp`. The portable copy replaces two compiler intrinsics with standard integer loops, preserving the mathematical checks. The separate implementation uses its own little-endian parsing, hash table, and local successor encoding. Both implementations nevertheless share the state model and capacity identity proved above; they provide an implementation cross-check, not two proofs with no shared mathematical assumptions.

Neither checker calls the generator, Z3, HiGHS, or another optimization solver. All mathematical checks use integers; floating point is used only for elapsed-time reporting. The single-task capacity tables are constructed by explicitly enumerating the earliest possible selection times, independently of the generator's capacity routine.

7.3 Complete verification counts and the lower bound

Two retained complete executions, one of the portable checker and one of the separate implementation, both passed with the following counts. No restriction was imposed on the first step.

Quantity	Count
Verified states	16,646,562
Labeled four-task choices	1,755,492,299
Choices covered by capacity	1,694,728,516
Choices covered by decreasing rank	60,763,783
Initial rank	75
Maximum recorded rank	75
Largest capacity horizon used	70

The capacity and rank counts sum to the total number of choices. For every transition not excluded by capacity, the successor was encoded and its smaller positive rank checked. The complete mathematical outputs, with timing fields omitted, and the hashes of the checker source files are retained in `verification/negative_full_results.json`. These are complete checks, rather than sampling results or solver infeasibility logs.

Lemma 3, together with $R(s_0) = 75$, rules out a 75-row schedule in the necessary relaxation. By (3), H_{75} has no packing 22-coloring. Therefore

$$\chi_{\mathrm{p}}(H_{75}) \geq 23. \tag{7}$$

8 An explicit periodic 23-color upper bound

The file `certificates/k6_23color_period480_certificate.json` contains the positive certificate: 480 rows, with the six actual vertex colors in each row ordered as (L, A, B, C, D, R). All entries are in $\{1, \ldots, 23\}$. Its SHA-256 is

`5e9136a6030a0827179c08d0a6b87efa1ec3606aabb561c429ad99068a337f31`.

Repeat the 480 rows periodically along the infinite chain. Each row has six distinct colors.

The checker `code/verify_periodic_certificate.py` uses the Python standard library. It constructs the adjacency list of 492 consecutive copies of K_6. It performs breadth-first search (BFS) from each of the 2880 vertices in the first period. It checks all vertex pairs whose later clique is between 1 and 12 cliques after the source clique, and compares the BFS distances with (1). The completed check reports

Quantity	Value
Status	PASS
Period	480
Number of colors	23
BFS sources	2880
BFS/formula comparisons	207,360
Same-color pairs in the BFS window	17,536
Successive cyclic same-color pairs	2880
Far-pair distance lower bound	25

The full output is retained in `outputs/period480_independent_check.json`.

This is a repeated straight chain. No edge is added from the last clique of a period back to its first clique. Pairs crossing a period boundary are checked in the explicitly constructed following cliques. For any pair in the infinite chain, translate the earlier clique by a multiple of 480 into the first period. If the clique-index difference is at most 12, the translated pair is among the checked pairs. If it is at least 13, then (1) gives

$$d(x, y) \geq 2 \cdot 13 - 1 = 25 > 23,$$

so every such pair automatically satisfies the packing condition. Pairs in a single clique are covered by the six distinct colors in every row. This proves that the finite window covers every possible conflict in the repeated coloring.

As an additional check, for each color i the minimum actual graph distance between consecutive cyclic occurrences is exactly $i + 1$, including occurrences across a period boundary. For a fixed earlier vertex, increasing the target clique index by one increases the distance in (1) by at least one. Thus nonconsecutive occurrences of the same color cannot be closer. These per-color checks are additional to, and do not replace, the complete-window BFS check.

The periodically repeated rows therefore give a packing 23-coloring of the infinite chain. Restricting to its first t cliques gives a packing coloring of every finite H_t, and hence

$$\chi_{\mathrm{p}}(H_t) \leq 23 \qquad (t \geq 1). \tag{8}$$

The first 75 rows and the complete edge list of H_{75} are also stored in `certificates/H75_graph_and_23color_witness.json`. The separate checker `code/verify_H75.py` confirms its 450 vertices and 1199 edges and performs 450 BFS runs. It checks all 101,025 unordered vertex pairs, including 8485 same-color pairs; all pass. This confirms the finite positive witness. The 22-color lower bound is established separately by the negative rank certificate.

Proof of Theorem 1. Combining (7) and (8) gives $\chi_{\mathrm{p}}(H_{75}) = 23$. For $t \geq 75$, the first 75 cliques induce an isometric copy of H_{75}: the bridge chain offers no shorter path through later cliques. A packing 22-coloring of H_t would therefore restrict to one of H_{75}, a contradiction. The uniform upper bound (8) proves $\chi_{\mathrm{p}}(H_t) = 23$ for every $t \geq 75$. Because a member of the family requires 23 colors, this uniform constant is sharp. □

9 Reproduction and verification scope

The proof depends on the mathematical reduction, the rank induction and the complete negative-certificate checks, together with the periodic positive certificate on actual vertex positions. The generator and its search decisions are not proof dependencies once the exact certificate is supplied. The additional Python sample checker is supplementary and cannot replace complete negative verification. File hashes identify the data being checked; they do not replace the mathematical checks.

Reproduction requires a 64-bit environment, Python 3.10 or later, and a C++17 compiler. The checkers use standard libraries and require no SAT or mixed-integer optimization solver. The full certificate and lookup table

require approximately 400 MB of memory, before environment-dependent overhead. The accompanying `REPRODUCIBILITY.md` gives integrity and decompression commands, compiler-specific commands, and expected verification counts. Download the compressed certificate asset matching the source package, verify it using `CERTIFICATE_ASSET.json`, and decompress it to `certificates/full_fast50m.bin`.

With a GCC- or Clang-compatible compiler available, the principal reproduction command from the package root is

```
python reproduce_primary.py
```

The wrapper checks the eight primary file hashes, compiles the original complete checker, verifies the entire negative certificate, checks the periodic and finite positive witnesses by BFS, and runs the supplementary Python sample check. It requires the stated complete counts and rejects a restricted first step. It creates a fresh `recheck/` directory; a repeat run can use a new `--out-dir`. Its compiler option uses GCC/Clang command-line syntax, not MSVC syntax.

The second complete implementation can be run separately:

```
g++ -O3 -std=c++17 code/independent_rank_check.cpp \
    -o independent_rank_check
./independent_rank_check certificates/full_fast50m.bin
```

For MSVC, the reproduction instructions give commands for both the separate implementation and the portable first checker. Each normal complete run must report success, initial rank 75, 16,646,562 states, and 1,755,492,299 labeled choices, with the capacity and rank counts stated above. The original checker's `--first-high` option belongs to a different exploratory certificate and must not be used for this theorem. The separate checker's `formal_metadata:true` field denotes its fixed certificate mode; it does not denote proof-assistant formalization. Its self-test and fixture modes likewise do not establish the theorem.

The positive checks can be run independently:

```
python code/verify_periodic_certificate.py \
    certificates/k6_23color_period480_certificate.json
python code/verify_H75.py \
    certificates/H75_graph_and_23color_witness.json
```

These establish upper bounds; the negative certificate is separately required for the lower bound.

The summaries in `verification/negative_full_results.json` and `outputs/` retain results of completed executions. Preparing this manuscript

does not itself constitute another complete negative-certificate run. Reproduction should check the actual certificate instead of treating a recorded PASS as a proof by itself. As in an ordinary computer-assisted proof, the checker implementations, their compilers and execution environments remain within the trust boundary. Hashes do not certify compiler semantics or the authenticity of unsigned execution records. No proof-assistant kernel formalization is provided, and no external expert-review endorsement is claimed.

The result does not determine the least counterexample length or all values for shorter chains. The computations for $t \leq 48$ reported in the original paper [1, Section 3.2.1, PDF p. 12] are consistent with a counterexample at $t = 75$; we make no assessment of unavailable original experimental data. No claim of worldwide priority is made.

Acknowledgment of AI assistance

This work was completed with AI assistance. Its mathematical claims rest on the arguments, exact certificates and reproducible checks presented here, rather than on generated text itself.

References

[1] Hanna Furmańczyk, Didem Gözüpek, and Sibel Özkan. *Packing coloring of graphs with long paths.* arXiv:2511.12761v1, 16 November 2025. `https://arxiv.org/abs/2511.12761v1`. Definition 2: PDF p. 3; Proposition 20: PDF p. 10; Section 3.2.1 and Conjecture 2: PDF p. 12.